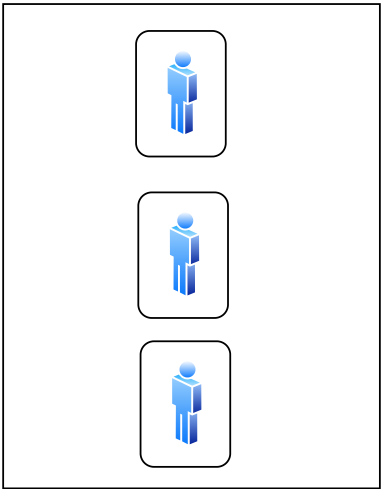
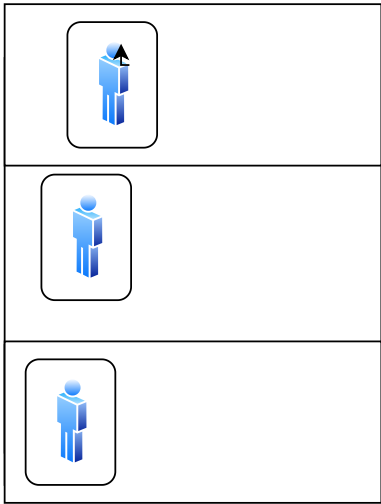


Docker

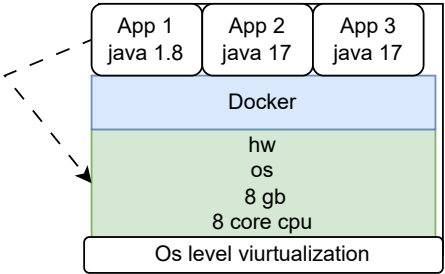
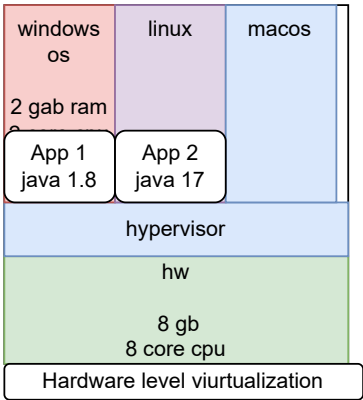
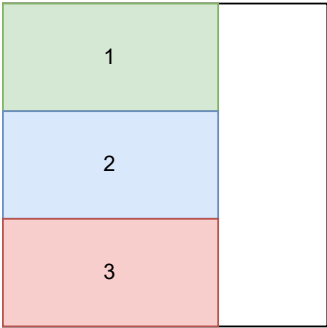
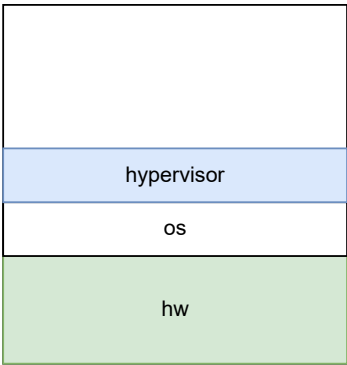
The whole purpose of containerization is isolation

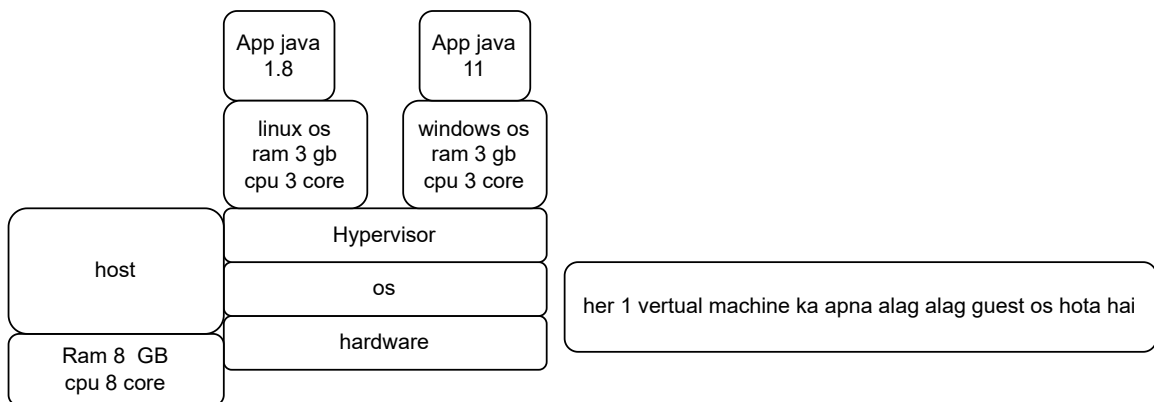
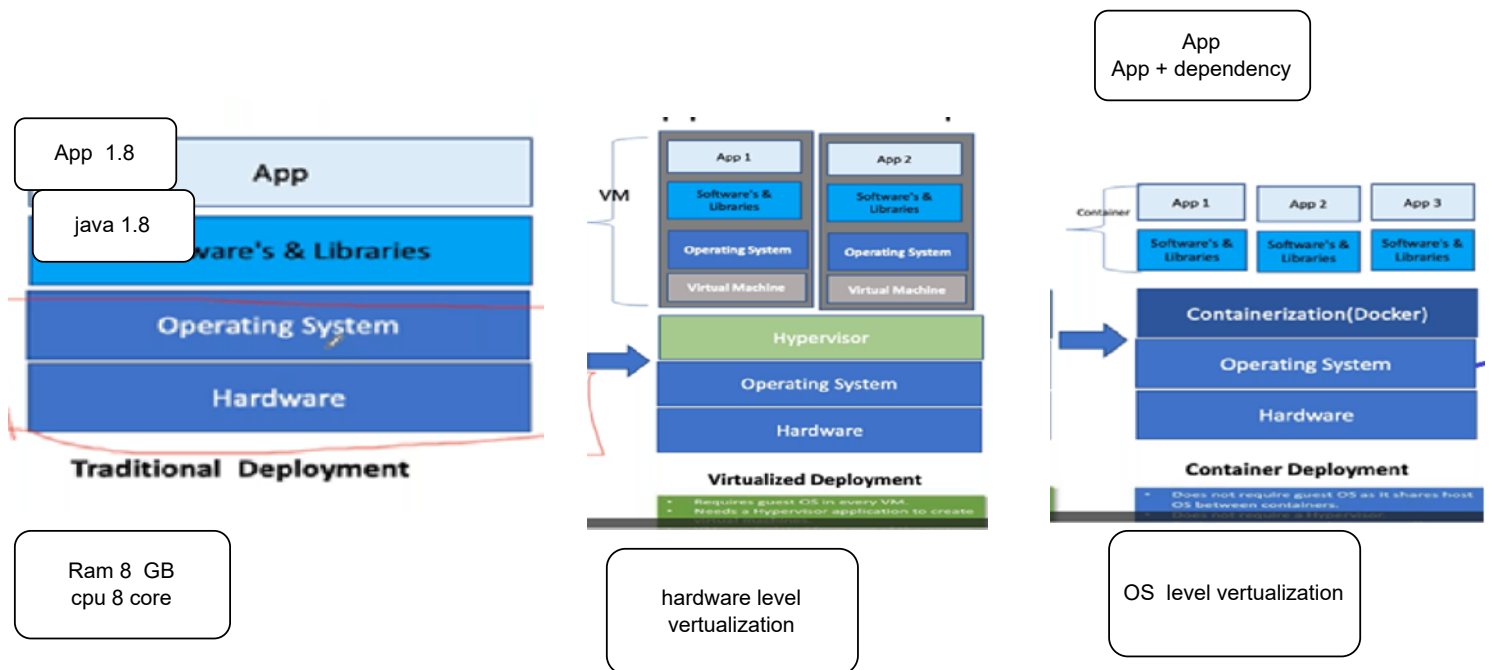


No Isolation

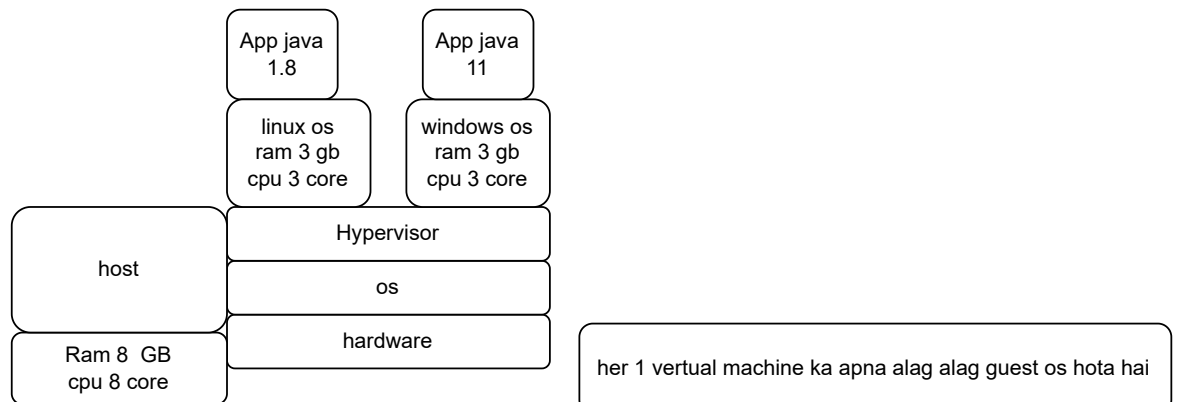


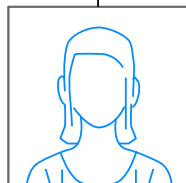
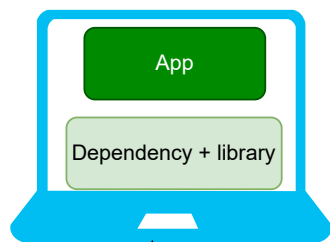
No Isolation



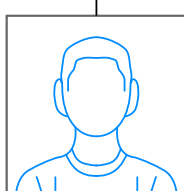
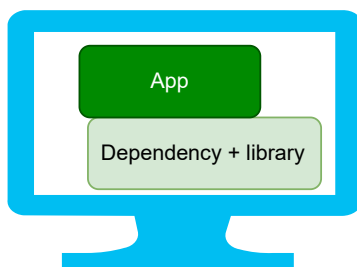


"Bare metal" का मतलब होता है सीधे असली हार्डवेयर पर काम करना, बिना किसी अतिरिक्त लेयर (जैसे operating system या virtualization) के।

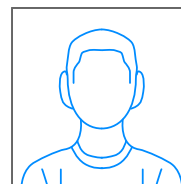




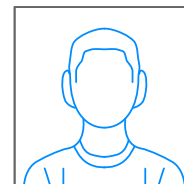
Chandni mam
developer



Abhay Bhai
operation



Ezaz Bhai
Dev + ope = Devops engg



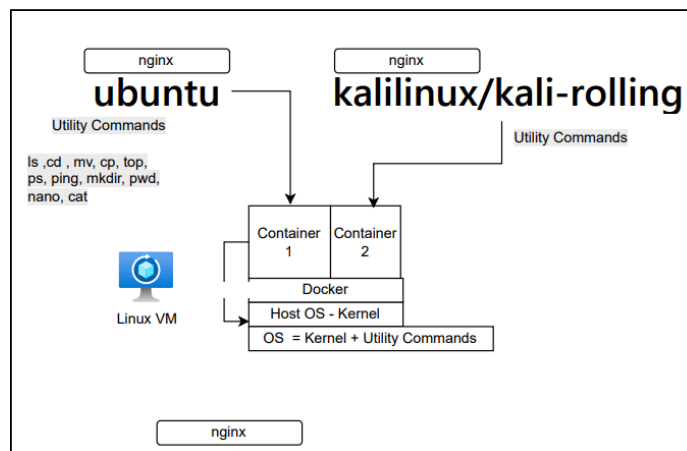
satyam Bhai
operation

Virtual Machine(application chلتا hai) ~ Docker
Container(application chلتا hai)

VM ke andar OS hota hai ~ Docker container ke andar OS nahi
hota hai..

VM ko banane ke lie ek image lgti hai...~ Docker Container ko
banane ke lie bhi ek image lagti hai..

VM ki image hoti hai vo Marketplace se aati hai..Aur agar image
docker container ki hai toh vo docker hub se aati hai..



Docker Architecture

Containerization

👉 App ko uski sari zaroori cheezon ke saath ek isolated package/container me
band kar dena taaki wo har system me same chale.

Container

Container contains everything which is required to run application

App + dependency ka package

Containerization software

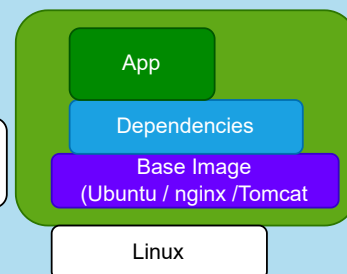
Docker , Rocket, Podman, Container-D, CRI-O

WHY Docker

Docker has very good CLI + API's to manager image or container

Docker

Docker 1 containerization software hai
jo ki app + dependency ka package bana deta hai
jo container ko chalane k lie required hoti hai

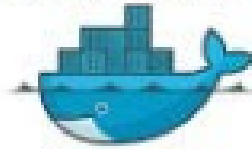


UNDERSTANDING DOCKER COMPONENTS

DOCKER DAEMON



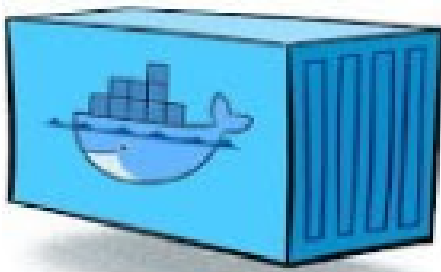
DOCKER CLIENT



DOCKER IMAGES



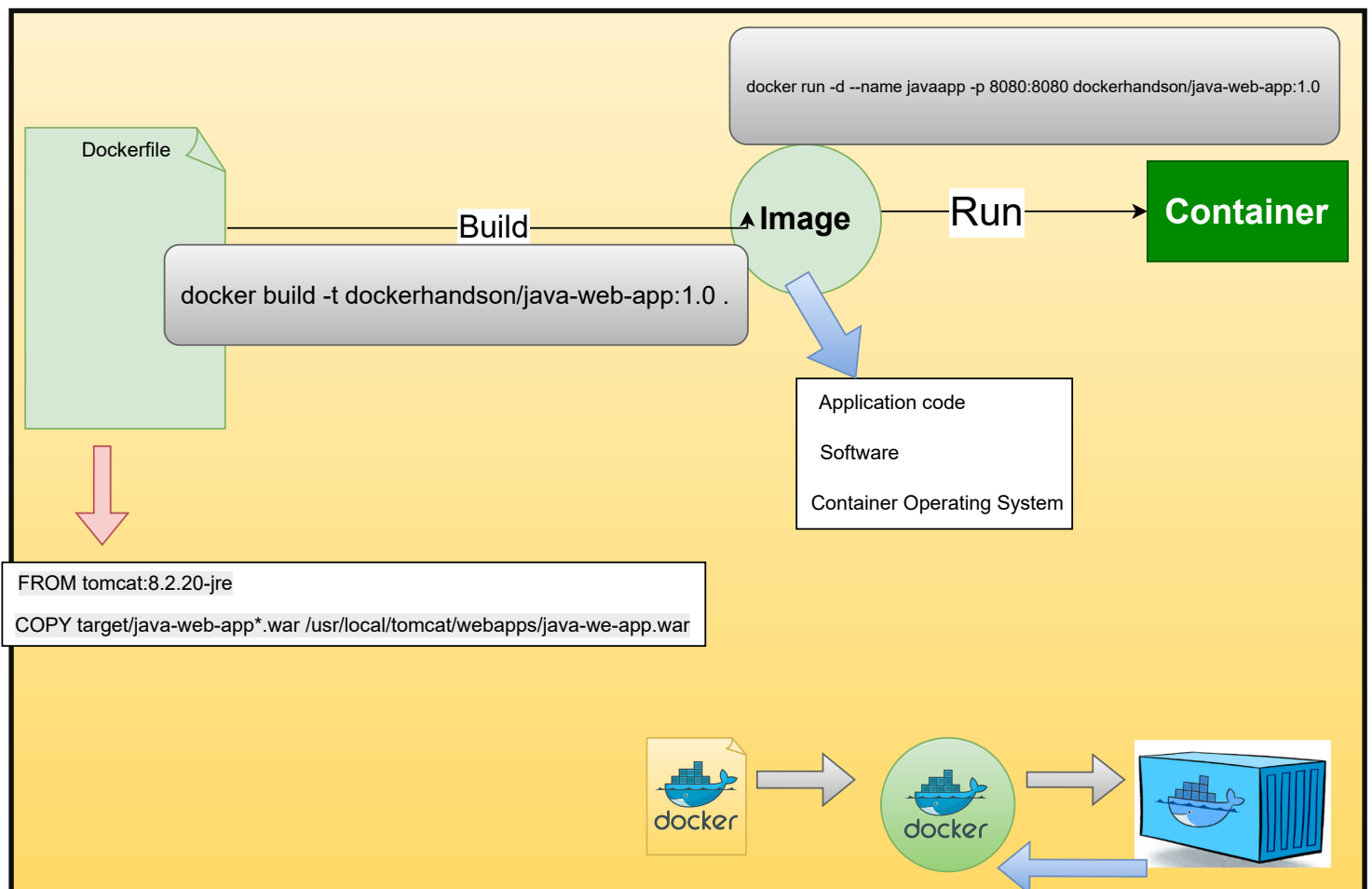
DOCKER CONTAINERS

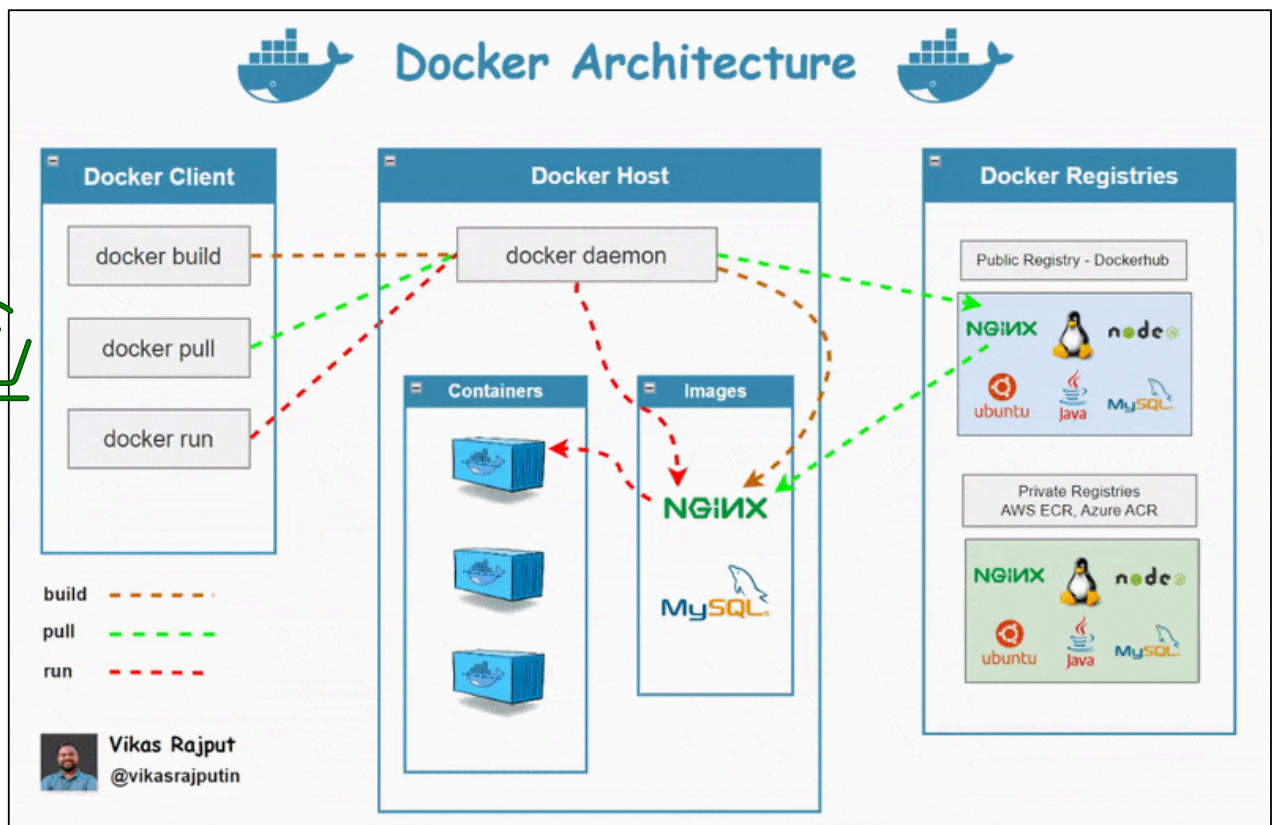
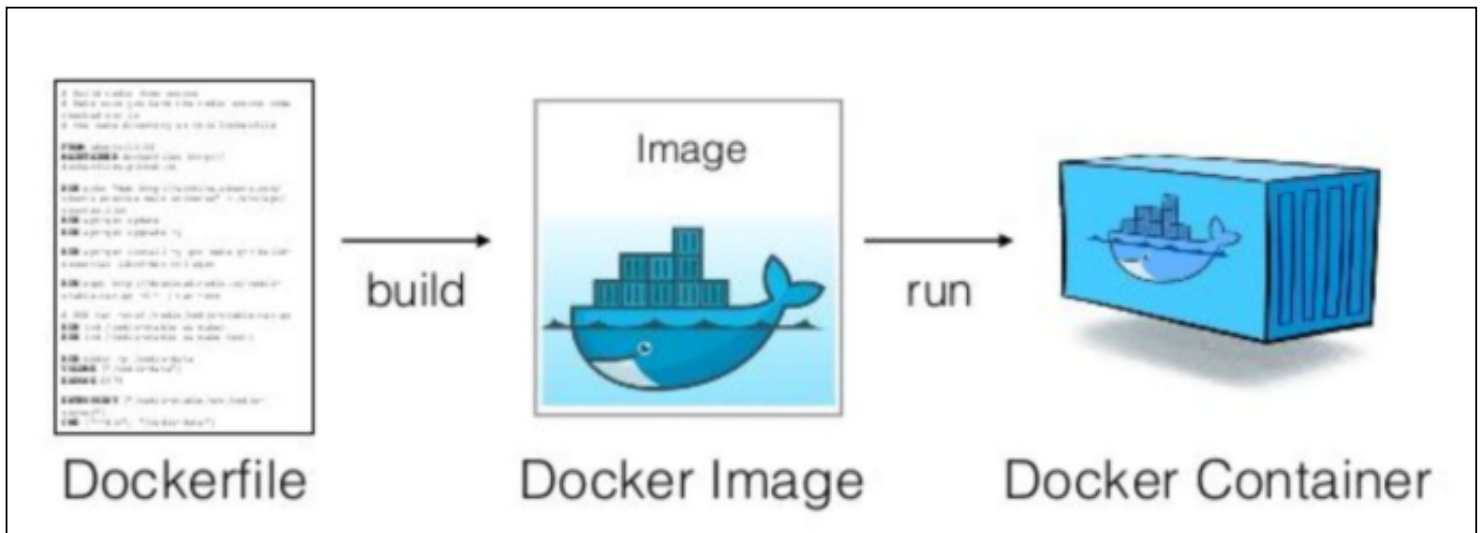


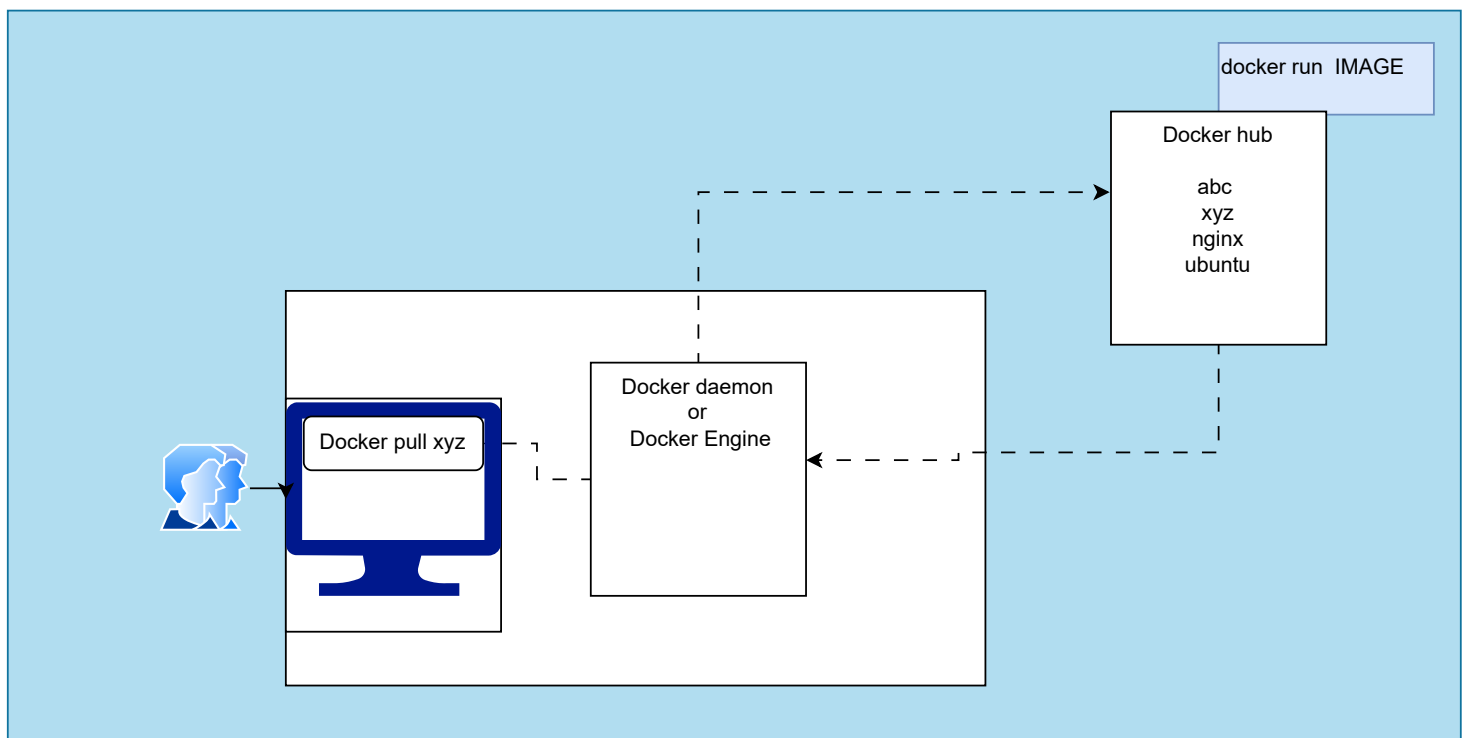
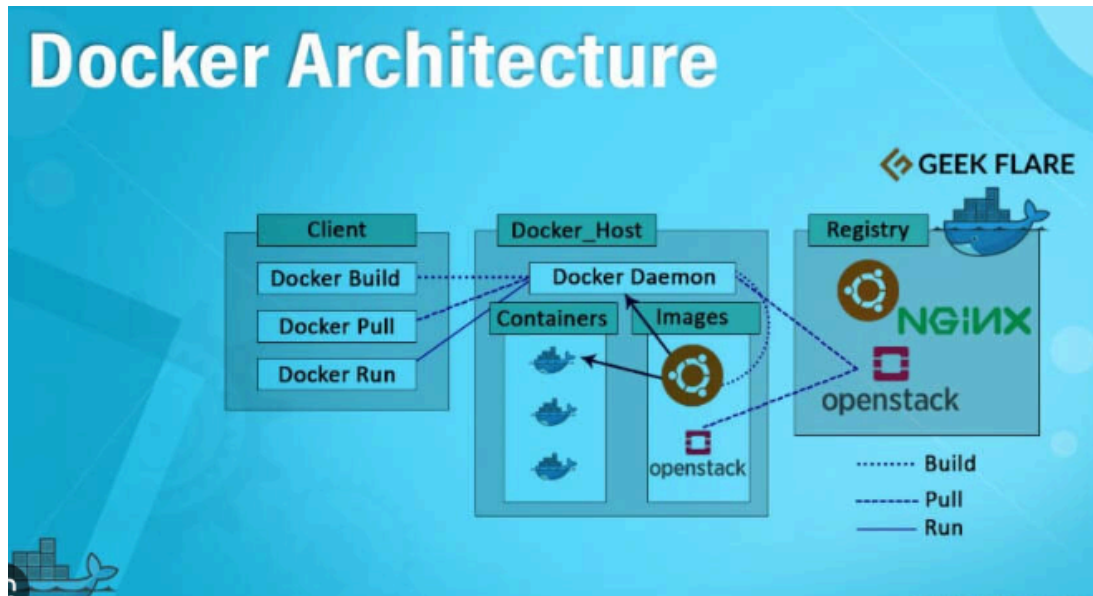
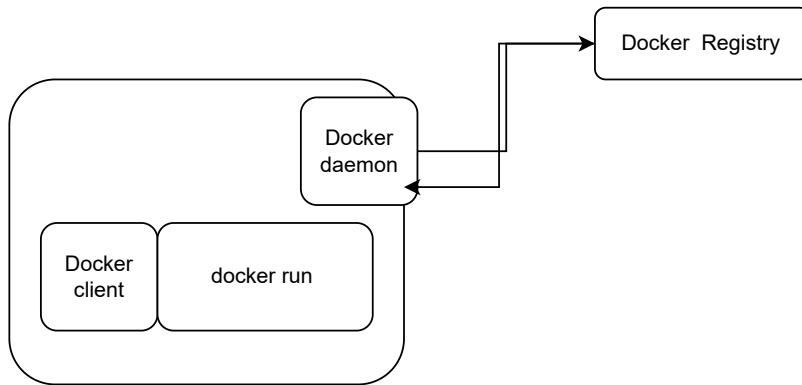
DOCKER REGISTRY

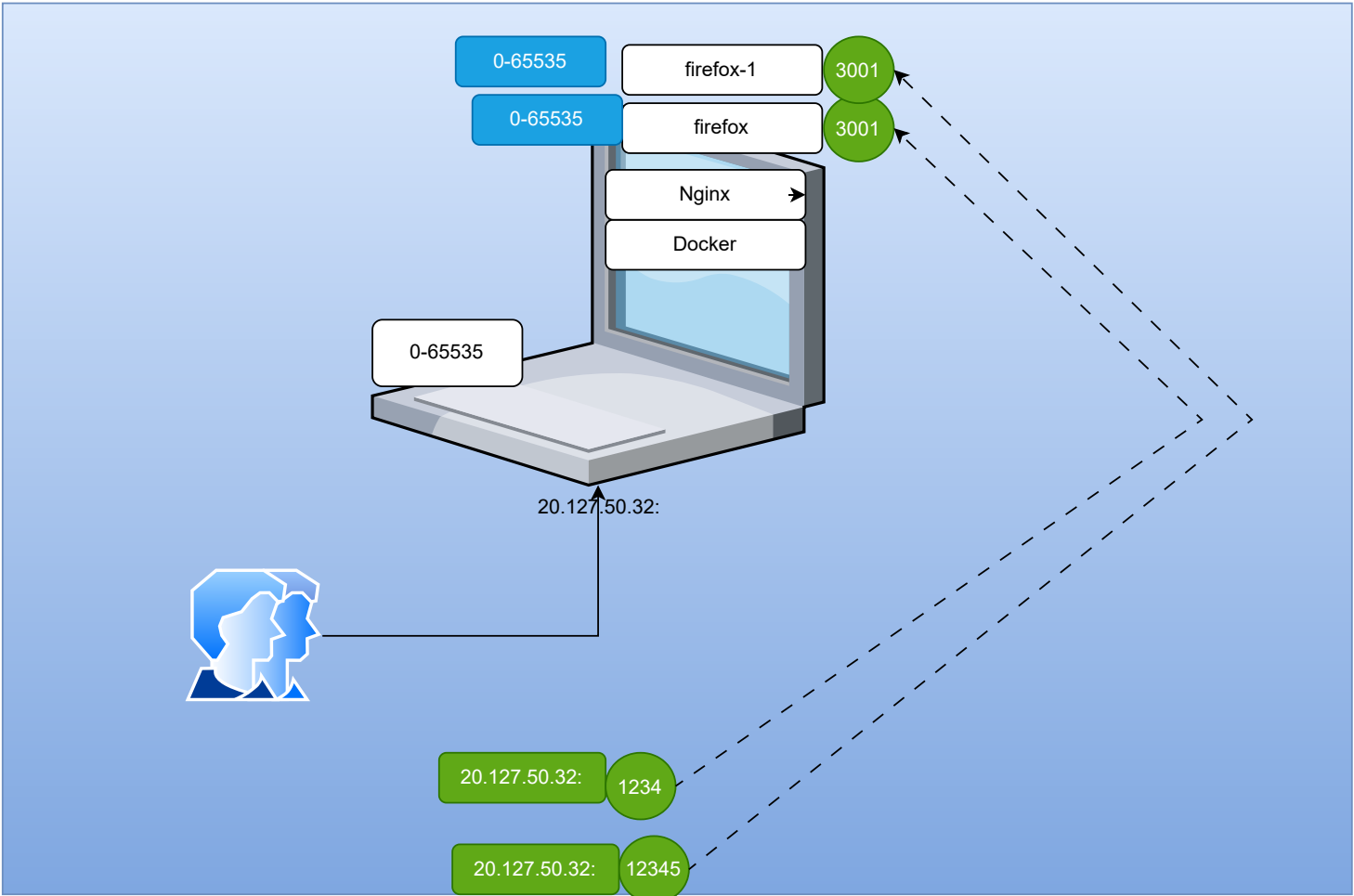
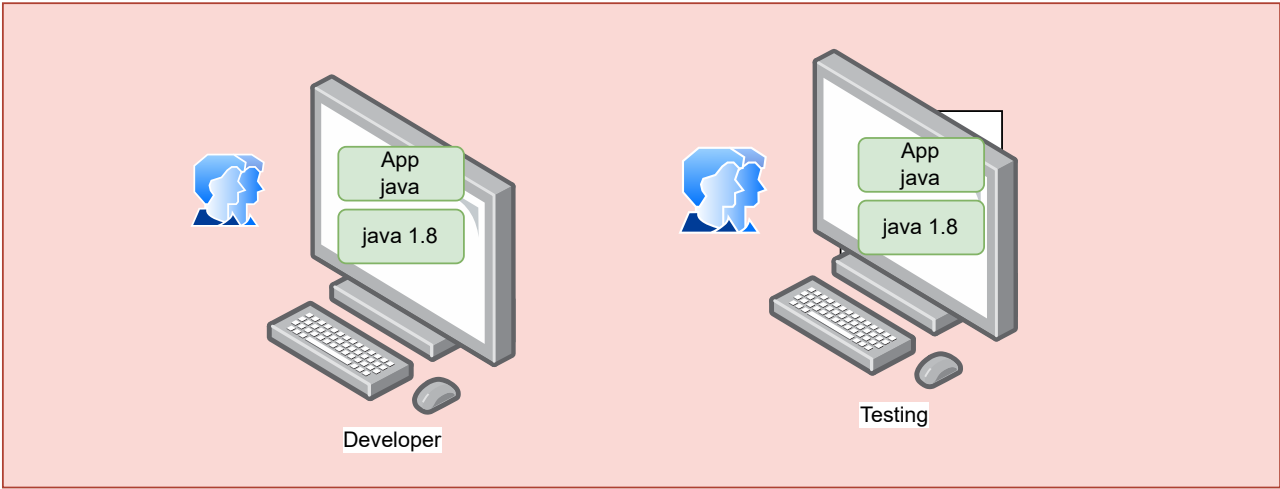


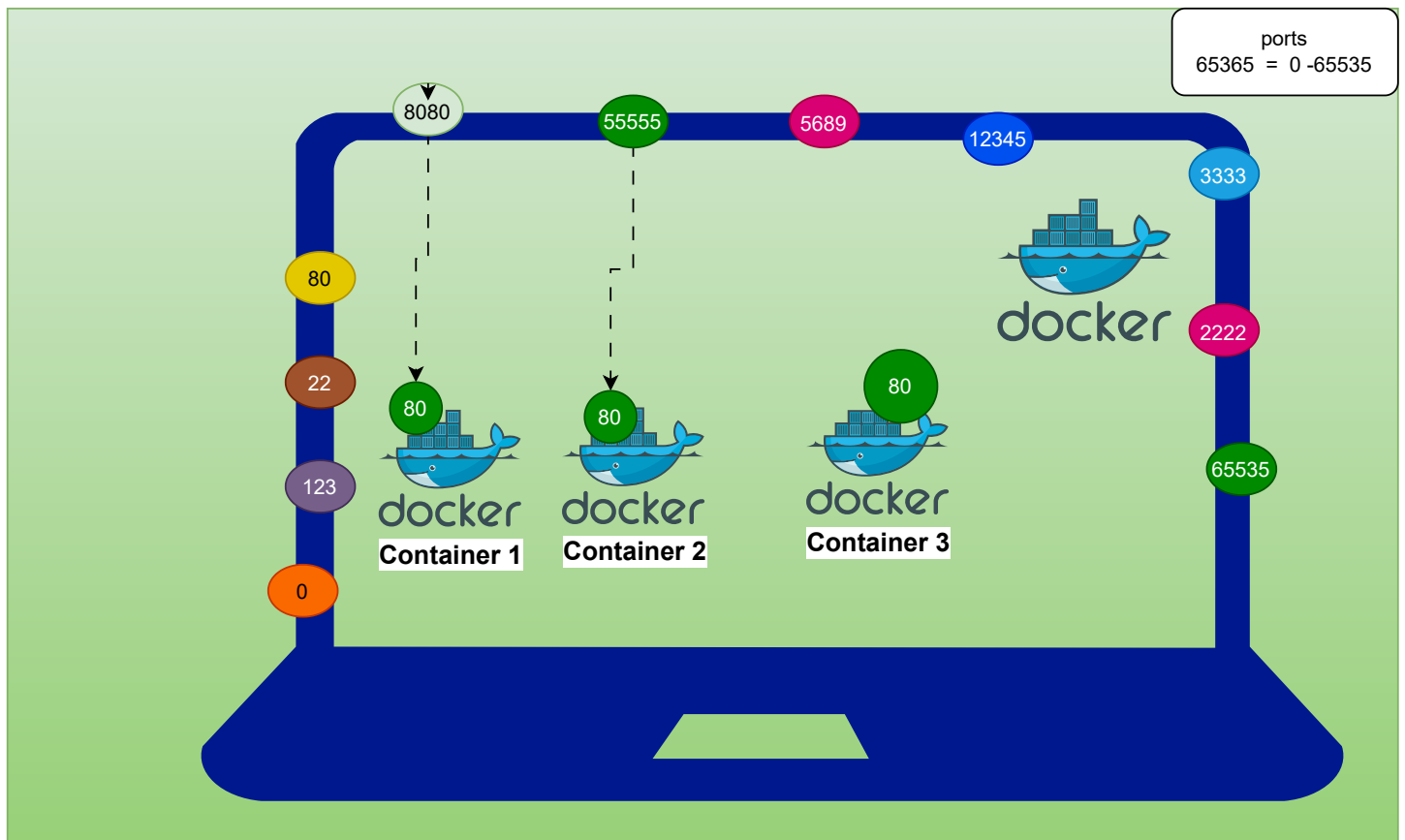
DOCKER SERVICES











```
docker run -d --name mehak -p 8080:80 nginx
```

```
docker run -d --name mehak2 -p 8081:80 nginx
```

<https://chatgpt.com/c/6a7b647c-6fd0-83ea-956c-a0c66b123ae0>

docker info

- 2 docker images
- 3 docker ps
- 4 docker ps -a
- 5 docker run hello-world
- 6 docker images
- 7 docker ps
- 8 docker ps -a
- 9 docker run nginx
- 10 docker images
- 11 docker run -d nginx
- 12 docker ps
- 13 docker exec b3e80865453a ls
- 14 docker exec b3e80865453a pwd
- 15 docker exec -it b3e80865453a -- bash
- 16 docker exec -it b3e80865453a --bash
- 17 docker exec -it b3e80865453a /bin/bash
- 18 docker ps -a
- 19 docker images
- 20 docker run -d --name arun-container -p 12345:80 nginx:latest
- 21 docker ps -a
- 22 which docker
- 23 which nginx
- 24 docker run -d --name firefox -p 1234:3001 linuxserver/firefox
- 25 docker ps
- 26 docker rm arun-container
- 27 docker rm -f arun-container
- 28 docker ps
- 29 docker rm busy_curie
- 30 docker stop busy_curie
- 31 docker ps
- 32 docker ps -a
- 33 docker rm busy_curie
- 34 docker ps
- 35 docker run -d --name firefox -p 12345:3001 linuxserver/firefox
- 36 docker run -d --name firefox-1 -p 12345:3001 linuxserver/firefox
- 37 docker ps -a
- 38 docker pull grafana/grafana
- 39 docker run -d --name=grafana -p 3000:3000 grafana/grafana
- 40 docker ps -a
- 41 docker ps
- 42 docker inspect b2f3e696bd14
- 43 docker run -d --name ezaz grafana/grafana
- 44 docker ps
- 45 docker inspect 1a206de08ed4
- 46 docker run -d --name monitoring -p 55555:3000 grafana/grafana
- 47 docker ps
- 48 docker inspect e1be1ad0ec50
- 49 history

1. BASIC COMMANDS



- 1 `docker --version` - Show Docker version
- 2 `docker info` - Display system-wide information
- 3 `docker help` - Show help for Docker
- 4 `docker login` - Login to Docker Hub
- 5 `docker logout` - Logout from Docker Hub

2. IMAGE COMMANDS



- 6 `docker pull <image>` - Pull an image from Docker Hub
- 7 `docker images` - List all local images
- 8 `docker rmi <image>` - Remove an image
- 9 `docker build -t <tag>` - Build an image from Dockerfile
- 10 `docker tag <image> <tag>` - Tag an image

3. CONTAINER COMMANDS (BASIC)



- 11 `docker run <image>` - Run a container
- 12 `docker run -it <image> <cmd>` - Run in interactive mode
- 13 `docker ps` - List running containers
- 14 `docker ps -a` - List all containers (all states)
- 15 `docker stop <container>` - Stop a running container
- 16 `docker start <container>` - Start a stopped container
- 17 `docker restart <container>` - Restart a container
- 18 `docker rm <container>` - Remove a container
- 19 `docker rm -f <container>` - Force remove a container
- 20 `docker container prune` - Remove stopped containers

7. DOCKER COMPOSE COMMANDS



- 37 `docker compose up` - Start all services
- 38 `docker compose up -d` - Start in detached mode
- 39 `docker compose down` - Stop and remove containers
- 40 `docker compose logs -f` - View logs of all services

COMMON OPTIONS



- `-d` Run in background
- `-it` Interactive terminal
- `-p` Publish port
- `-v` Mount volume
- `--name` Assign container name
- `--rm` Remove container

QUICK TIPS



- Use official images
 - Keep images small
 - Use `.dockerignore`
 - Don't run as root (if possible)
 - Clean up unused resources
- `docker system prune -a`

CONTAINER vs IMAGE



IMAGE

- Read-only template
- Built from Dockerfile
- Stored in Docker Registry
- Don't have state



CONTAINER

- Runnable instance of image
- Has a writable layer
- Runs processes
- Has state

COMMON USE CASES



- Web Applications
- Microservices
- CI/CD Pipelines
- Databases
- Testing Environments

REMEMBER

- Build → Ship → Run
- Automate Everything
- Monitor & Optimize
- Security First



Docker powers modern DevOps!
Learn it deeply. Use it daily.
Master it. Become unstoppable!



Let's connect!



@uxzwal

CHEAT SHEET

40

MOST USED

DOCKER

COMMANDS

FOR DEVOPS ENGINEERS



4. CONTAINER COMMANDS (ADVANCED)



- 21 `docker logs <container>` - View container logs
- 22 `docker logs -f <container>` - Follow container logs
- 23 `docker exec -it <container> <cmd>` - Execute command in container
- 24 `docker inspect <container>` - Inspect container details
- 25 `docker stats <container>` - Show live container stats
- 26 `docker top <container>` - Show running processes
- 27 `docker cp <src> <container>:<dest>` - Copy files to container
- 28 `docker cp <container>:<src> <dest>` - Copy files from container

5. NETWORK COMMANDS

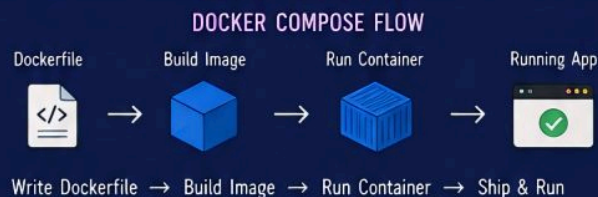


- 29 `docker network ls` - List all networks
- 30 `docker network create <name>` - Create a network
- 31 `docker network inspect <name>` - Inspect a network
- 32 `docker network rm <name>` - Remove a network

6. VOLUME COMMANDS



- 33 `docker volume ls` - List all volumes
- 34 `docker volume create <name>` - Create a volume
- 35 `docker volume inspect <name>` - Inspect a volume
- 36 `docker volume rm <name>` - Remove a volume



```
root@ubuntu:~$ docker --help
Usage: docker [OPTIONS] COMMAND
```

A self-sufficient runtime for containers

Common Commands:

run	Create and run a new container from an image
exec	Execute a command in a running container
ps	List containers
build	Build an image from a Dockerfile
bake	Build from a file
pull	Download an image from a registry
push	Upload an image to a registry
images	List images
login	Authenticate to a registry
logout	Log out from a registry
search	Search Docker Hub for images
version	Show the Docker version information
info	Display system-wide information

Management Commands:

builder	Manage builds
buildx*	Docker Buildx
container	Manage containers
context	Manage contexts
image	Manage images
manifest	Manage Docker image manifests and manifest lists
network	Manage networks
plugin	Manage plugins
system	Manage Docker
trust*	Manage trust on Docker images
volume	Manage volumes

Swarm Commands:

swarm	Manage Swarm
-------	--------------

Commands:

attach	Attach local standard input, output, and error streams to a running container
commit	Create a new image from a container's changes
cp	Copy files/folders between a container and the local filesystem
create	Create a new container
diff	Inspect changes to files or directories on a container's filesystem
events	Get real time events from the server
export	Export a container's filesystem as a tar archive
history	Show the history of an image
import	Import the contents from a tarball to create a filesystem image
inspect	Return low-level information on Docker objects
kill	Kill one or more running containers
load	Load an image from a tar archive or STDIN
logs	Fetch the logs of a container
pause	Pause all processes within one or more containers
port	List port mappings or a specific mapping for the container
rename	Rename a container
restart	Restart one or more containers
rm	Remove one or more containers
rmi	Remove one or more images
save	Save one or more images to a tar archive (streamed to STDOUT by default)
start	Start one or more stopped containers
stats	Display a live stream of container(s) resource usage statistics
stop	Stop one or more running containers
tag	Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
top	Display the running processes of a container
unpause	Unpause all processes within one or more containers
update	Update configuration of one or more containers
wait	Block until one or more containers stop, then print their exit codes

wait Block until one or more containers stop, then print their exit codes

Global Options:

--config string Location of client config files (default "/root/.docker")
-c, --context string Name of the context to use to connect to the daemon (overrides DOCKER_HOST env var and default context set with "docker context use")
-D, --debug Enable debug mode
-H, --host string Daemon socket to connect to
-l, --log-level string Set the logging level ("debug", "info", "warn", "error", "fatal") (default "info")
--tls Use TLS; implied by --tlsverify
--tlscacert string Trust certs signed only by this CA (default "/root/.docker/ca.pem")
--tlscert string Path to TLS certificate file (default "/root/.docker/cert.pem")
--tlskey string Path to TLS key file (default "/root/.docker/key.pem")
--tlsverify Use TLS and verify the remote
-v, --version Print version information and quit

Run 'docker COMMAND --help' for more information on a command.

For more help on how to use Docker, head to <https://docs.docker.com/go/guides/>

docker [OPTIONS] COMMAND

docker COMMAND

docker run

docker build

docker pull

docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

docker run IMAGE

docker run IMAGE

docker run grafana/grafana

small : mandatory

capital : variable

[] : optional

docker [OPTIONS] COMMAND

docker run

docker run IMAGE

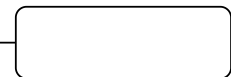
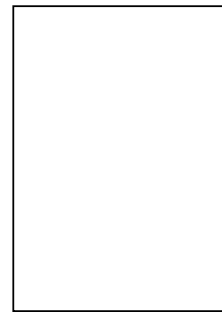
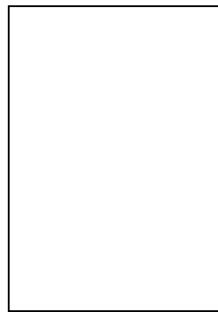
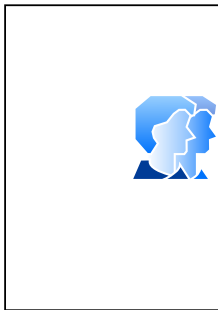
docker run ubuntu



Actor

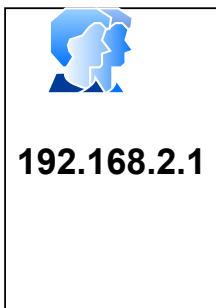


Vnet - A 192.168.1.0/24 : 256 ip

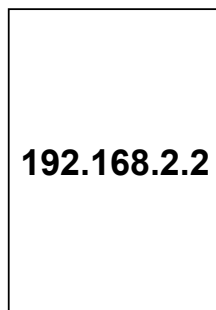


veering

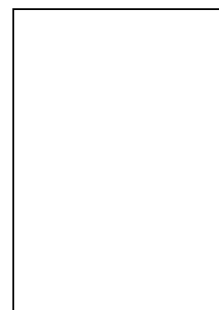
Vnet - B 192.168.2.0/24 : 256 ip



192.168.2.1



192.168.2.2



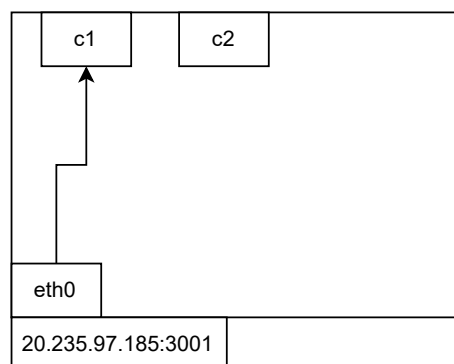
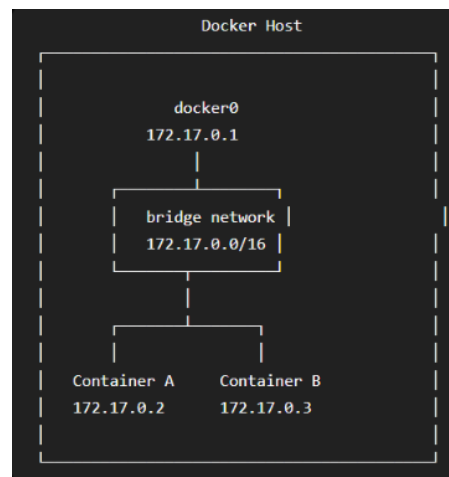
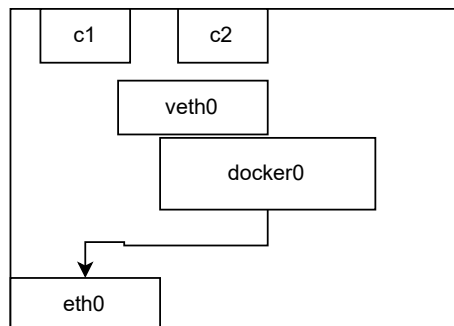
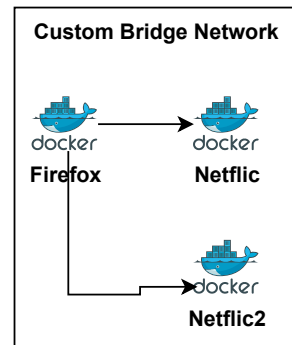
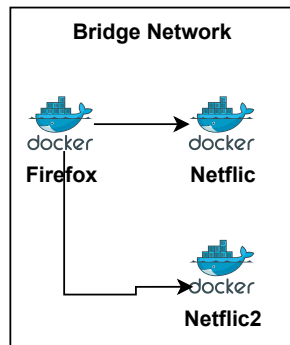
Ezaz@docker-vm:~\$ docker network --helpUsage: docker network

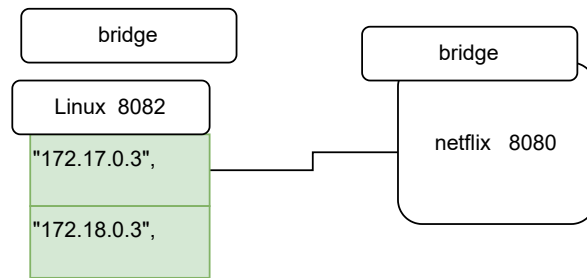
COMMANDManage networksCommands:

connect Connect a container to a network create Create a network disconnect

Disconnect a container from a network inspect Display detailed information on one or more networks ls List networks prune Remove all unused networks rm Remove one or more networks

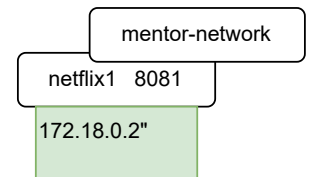
docker network create
docker network connect
docker network disconnect
docker network ls
docker network inspect
docker network prune
docker network rm



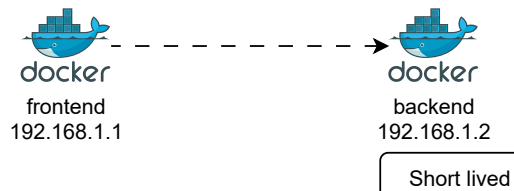


==> curl -v 172.17..0.2:5000/

==> curl -v netflix1:80



Bridge (ip se hit maar sakte hai)



Custom Bridge (ip aur container_Name se hit maar sakte hai)

